

Lógica de programação

O

João Augusto T. Laner

Unisenac
Centro Universitário RS



Fecomércio
IFEP
Sesc
Sindicatos Empresariais





GIT

Versionamento de arquivos

- Surgiu como ferramenta para otimizar o desenvolvimento do kernel do Linux;
- Faz parte da comunidade Open Source*;
- Ajuda a manter um *backup*;
- Padrão em sistemas UNIX/LINUX;
- Aplicação do terminal;

*Open Source: Software livre, livre das amarras comerciais. (MIT, GPL3, CC-BY)

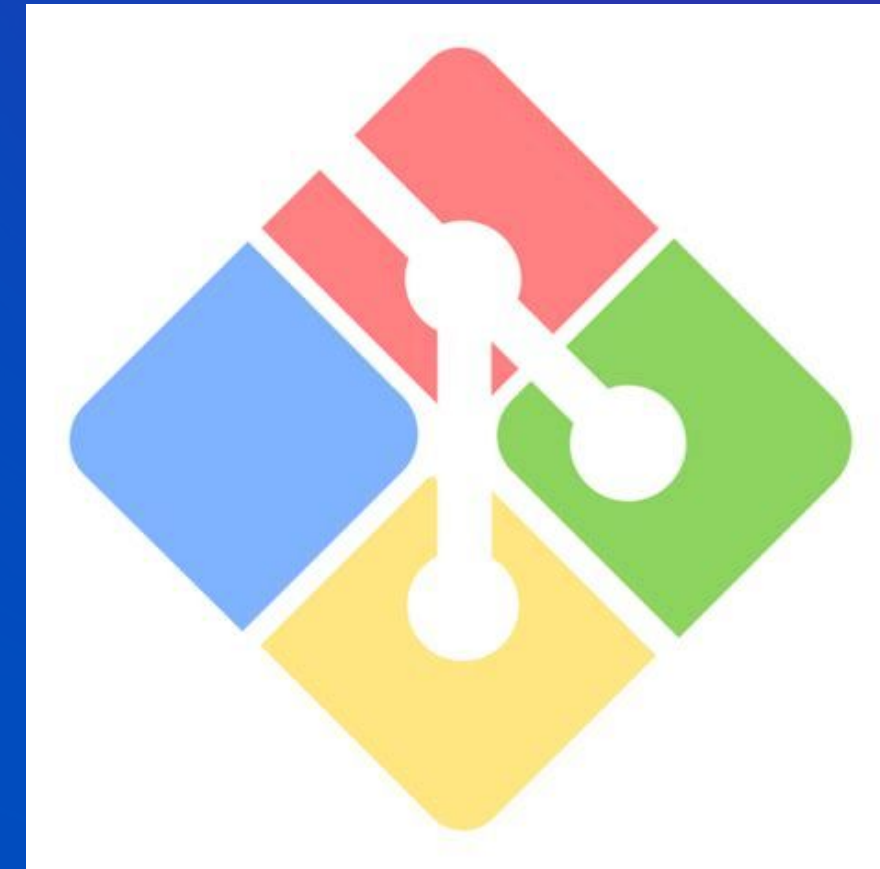


2026

GIT *Bash*

Aplicação

- Em sistemas windows, que são mais voltados a usuários finais, não existe um terminal com capacidade de gerenciar todo o sistema. Por isso, para usarmos o **GIT** aqui, instalaremos a aplicação *Git Bash*.

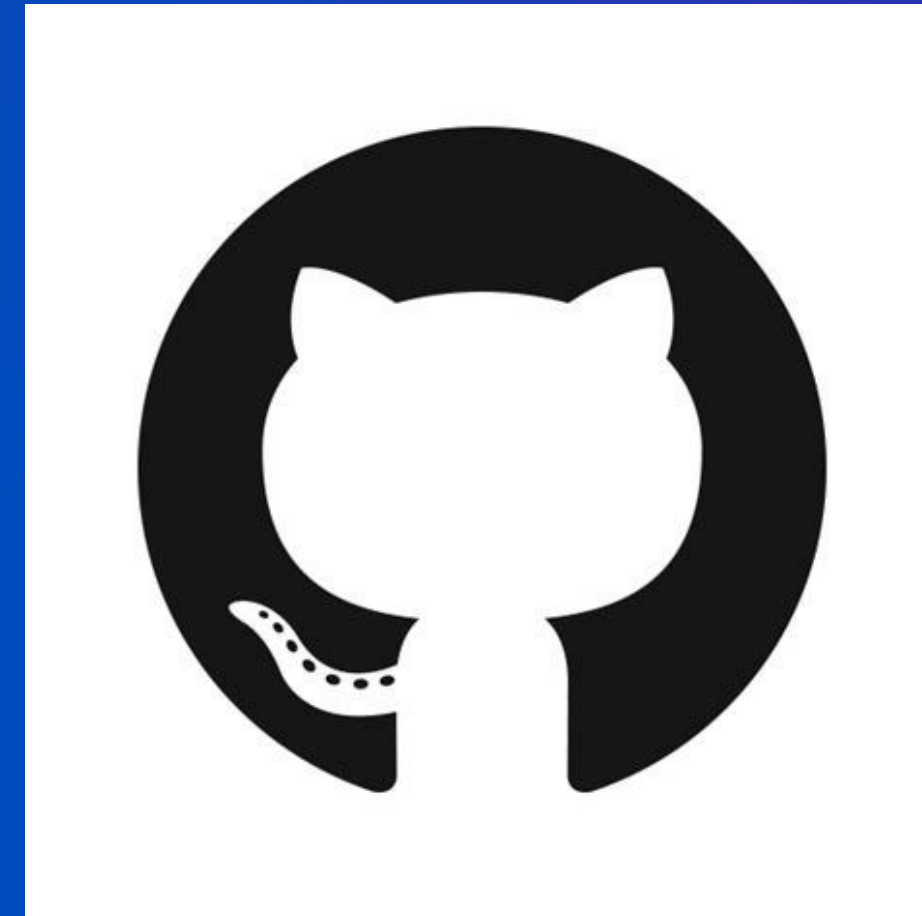




2026

Token (senha)

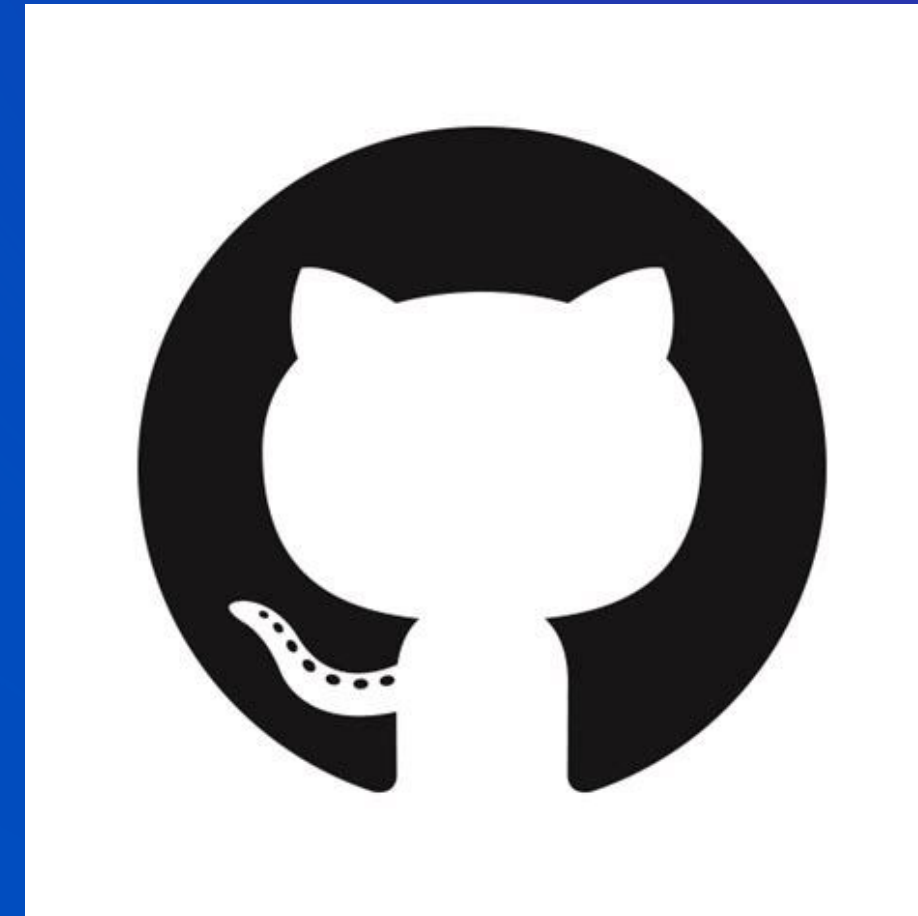
- A plataforma do git aboliu em 2021, o uso da **autenticação** por senha, então usaremos a nova forma fornecida.





Token (senha)

- Passo a passo:
 1. Entrar no **Github**
 2. Ir nas configurações do usuário
(clicar no usuário no canto superior direito)
 3. Na aba à esquerda, clicar em:
*developer tools > tokens (classic) >
generate new token*





SSH (Secure shell)

- Passo a passo:
 1. Entrar no **Github**
 2. Ir nas configurações do usuário
(clicar no usuário no canto superior direito)
 3. Na aba à esquerda, clicar em:

SSH > New SSH key

Depois, no terminal;

```
ssh-keygen -t rsa -b 4096 -f ~/<path_to_file>
```







Precauções:

Os arquivos confidenciais, ou senhas, **jamais** podem ser guardados em **repositórios públicos** no GIT.

Use um gerenciador de senhas!!!

(LastPass, **ProtonPass**, 1Password, BitWarden)



Comandos úteis do git:

git add (adiciona os arquivos ao *commit* que será feito)

git commit (salva as mudanças no histórico do git)

git push (manda o commit para o servidor)

git pull (puxa alterações do servidor, caso tenha novas alterações)

git log (mostra todos os registros de commits já realizados com seus identificadores)

git status (mostra os status das mudanças no terminal (indica quais arquivos foram alterados))

Markdown!

João Augusto T. Laner

Unisenac
Centro Universitário RS



Fecomércio
IFEP
Sesc
Sindicatos Empresariais



2026

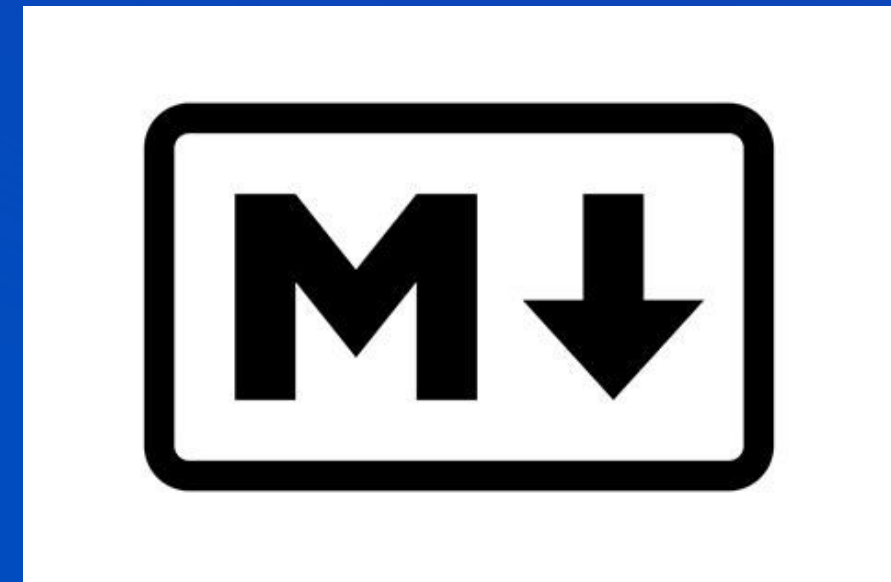
Markdown

Linguagem de marcação

Usaremos **markdown (.md)**, para criar nossos documentos, pois com eles podemos estruturar textos com comandos simples.

Para uma referência completa, consulte:

Markdown Cheat Sheet



Markdown:

Títulos (usamos hashtags para criar títulos (quanto mais hashtags MENOR a importância do título))

-/*/1. listas (hífens e asteriscos indicam listas sem ordem, já números indicam ordem)

> [!NOTE/IMPORTANT/WARNING/DANGER] (indica um bloco de informação que merece destaque)

para maiores informações, consultar: **Markdown.org**

Lógica de Programação

João Augusto T. Laner

Unisenac
Centro Universitário RS



Fecomércio
IFEP
Sesc
Sindicatos Empresariais

História

João Augusto T. Laner

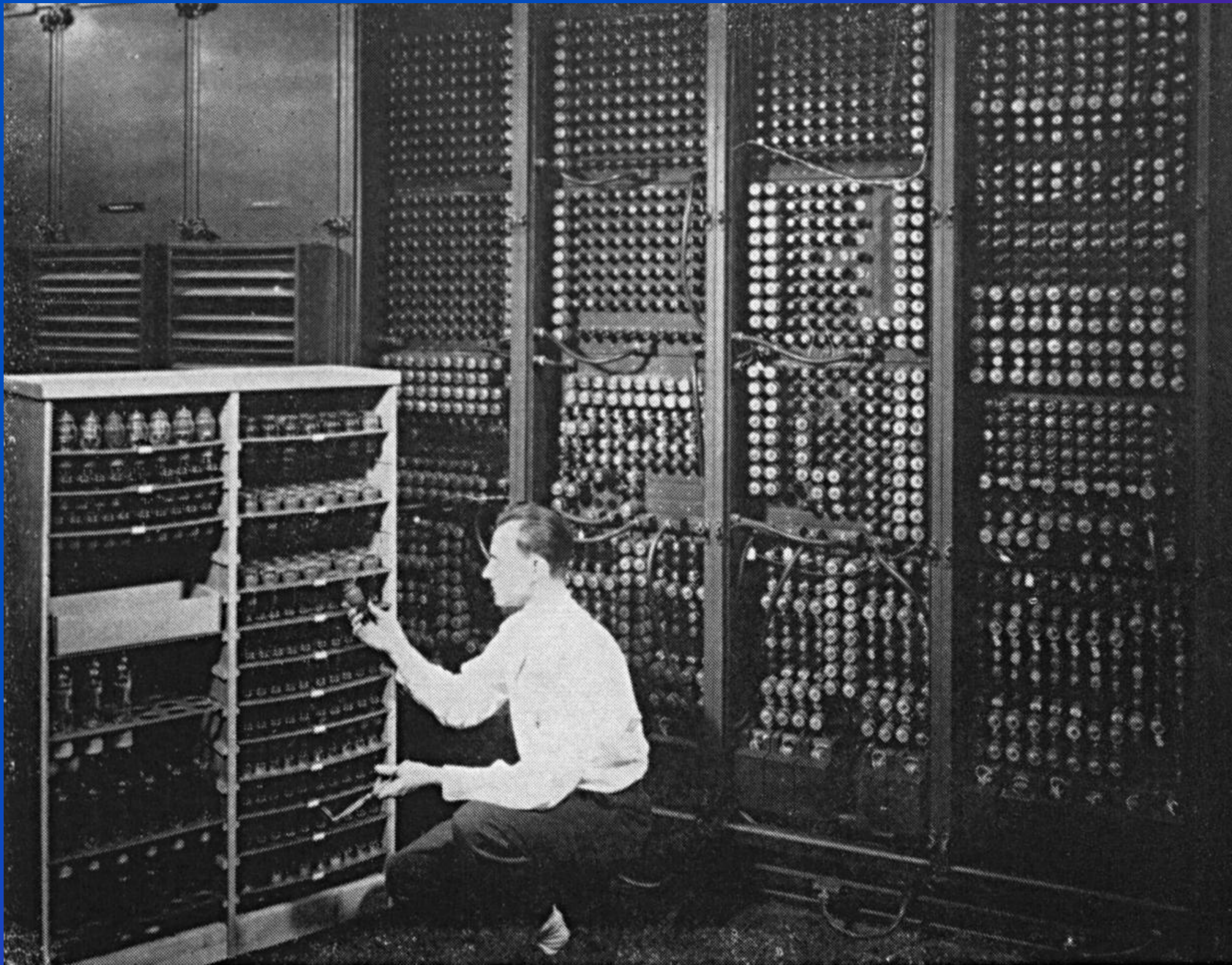
Unisenac
Centro Universitário RS



Fecomércio
IFEP
Sesc
Sindicatos Empresariais

ENIAC

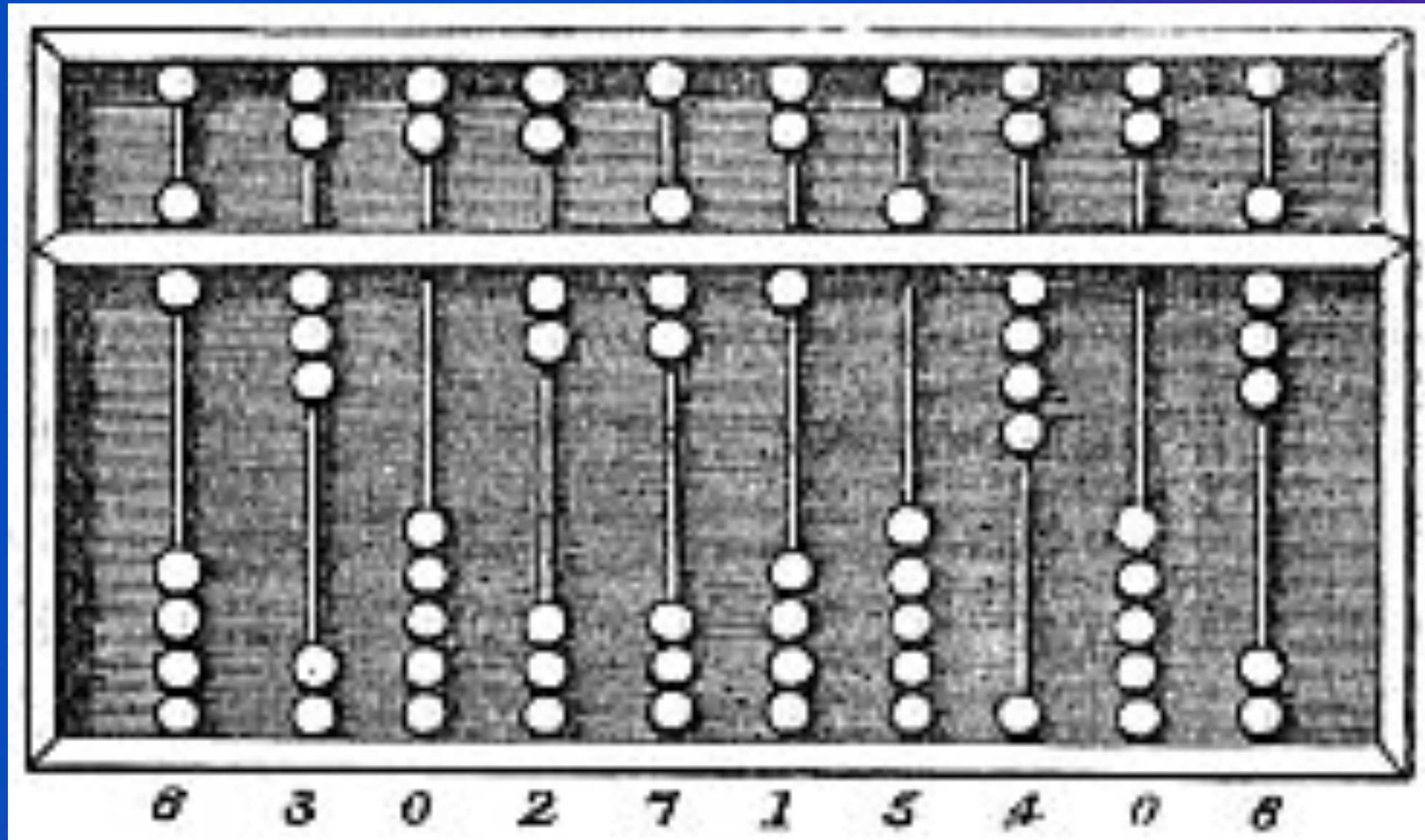
2026



By M. Weik - <https://ftp.arl.army.mil/ftp/historic-computers/> ; Transferred from pl.wikipedia to Commons by Masur using CommonsHelper., Public Domain, <https://commons.wikimedia.org/w/index.php?curid=159291086>

ÁBACO

2026



By Encyclopædia Britannica - Article for "abacus", 9th edition Encyclopedia Britannica, volume 1 (1875); scanned and uploaded by Malcolm FarmerTransferred from en.wikipedia to Commons., Public Domain, <https://commons.wikimedia.org/w/index.php?curid=146649>



2026

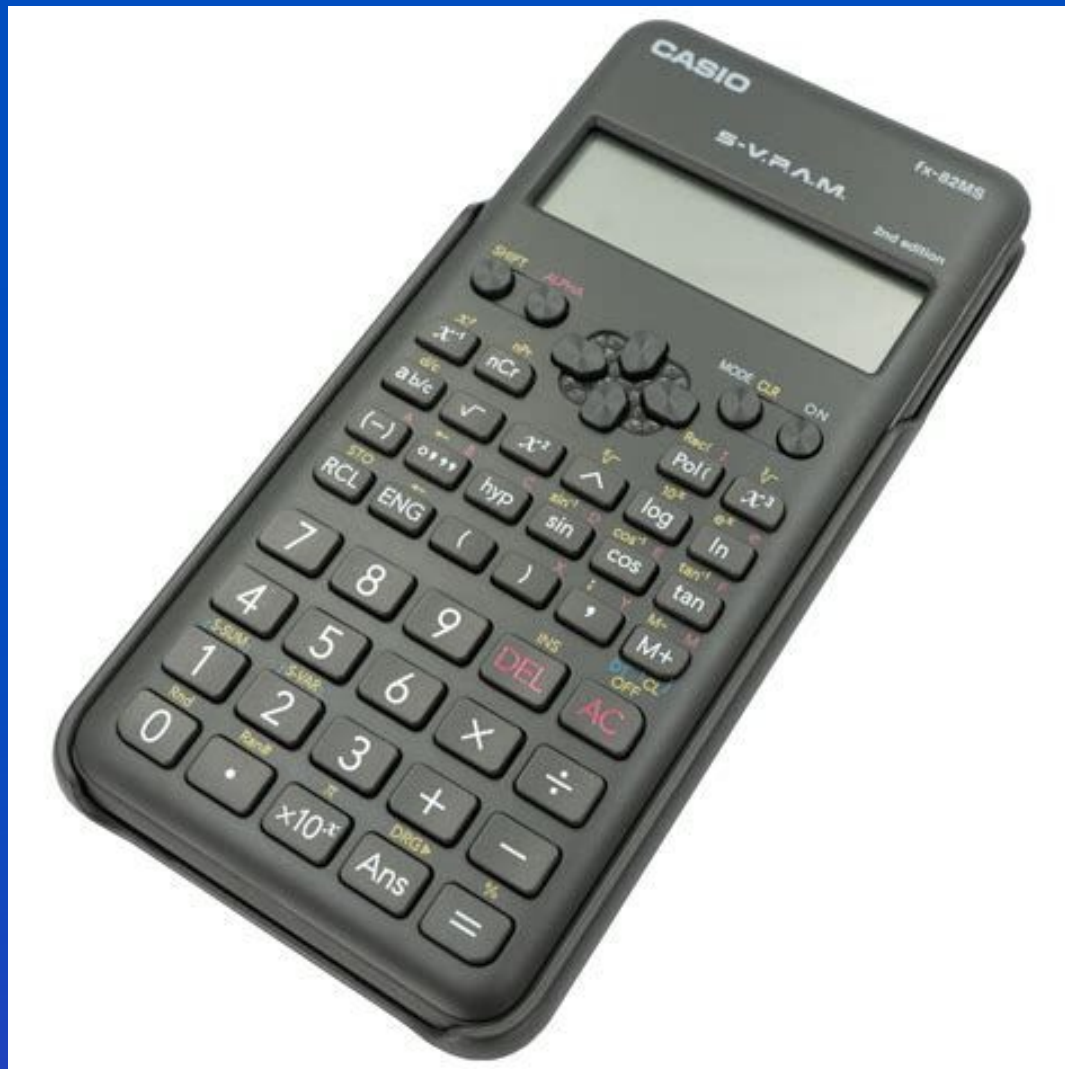
O que são Computadores? Nos dias atuais?

São calculadoras **MUITO, MUITO, MUITO** rápidos.



2026

Se são calculadoras, então...



=





2026

Sim! E não...

O que a calculadora tem?

Tem teclado?



2026

Sim! E não...

O que a calculadora tem?

Tem teclado? **Sim**



Sim! E não...

O que a calculadora tem?

Tem teclado? **Sim**

Tem memória?



Sim! E não...

O que a calculadora tem?

Tem teclado? **Sim**

Tem memória? **Sim**



Sim! E não...

O que a calculadora tem?

Tem teclado? **Sim**

Tem memória? **Sim**

Tem processamento?



Sim! E não...

O que a calculadora tem?

Tem teclado? **Sim**

Tem memória? **Sim**

Tem processamento? **Mais ou menos**



Uma super Um pouco da arquitetura

Temos vários níveis de memórias: **Disco (HD), RAM, L1, L2, L3**

Processadores que podem executar mais de 10^6
instruções por segundo

AMD Ryzen Threadripper PRO 9995WX



2026

Description:

Class: Desktop, Server

Socket: sTR5

Clockspeed: 2.5 GHz

Turbo Speed: 5.4 GHz

Cores: 96 **Threads:** 192

Typical TDP: 350 W

Cache per CPU Package:

L1 Instruction Cache: 96 x 32 KB

L1 Data Cache: 96 x 48 KB

L2 Cache: 96 x 1024 KB

L3 Cache: 384 MB



Instruções de alto nível

Operações aritméticas

adição (+) $\rightarrow 2 + 2 = 4$

subtração (-) $\rightarrow 3 - 2 = 1$

multiplicação (*) $\rightarrow 3 * 20 = 60$

divisão (/) $\rightarrow 77 / 7 = 11$

módulo (%) $\rightarrow 26 \mid_5 = 1$

Algoritmos

e o bolo...

João Augusto T. Laner

Unisenac
Centro Universitário RS



Fecomércio
IFEP
Sesc
Sindicatos Empresariais



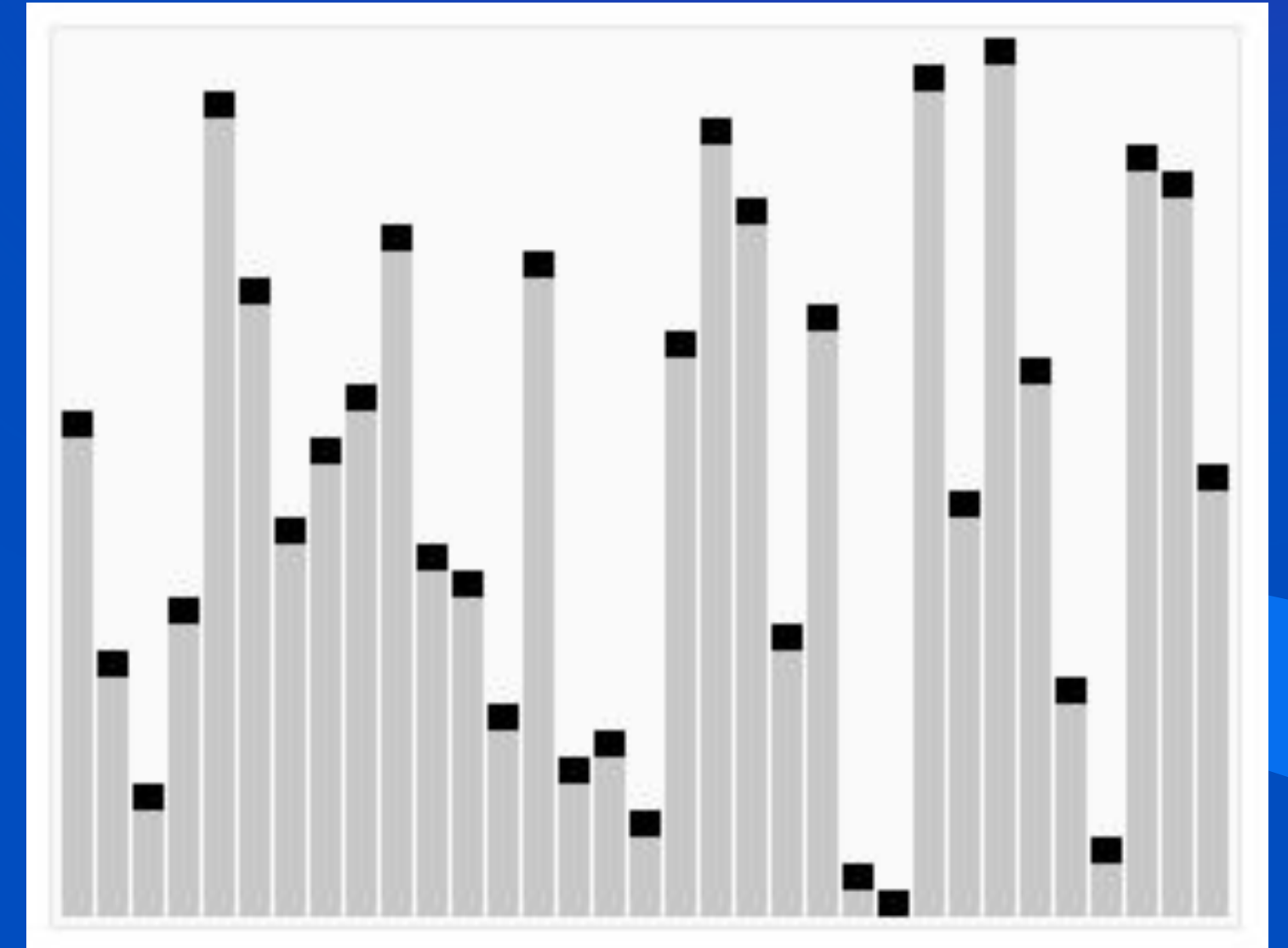
2026

Algoritmos

O que são?

Uma sequência de ações!

Algoritmos





Algoritmos

Receita do bolo:

- Misturar farinha, bicarbonato, amido de milho e fermento; (secos)
- Misturar ovos, açúcar, chocolate em pó, óleo, água quente e bater até ficar homogêneo;
- Incorpora os "secos" na mistura com os ovos;
- Leva ao forno preaquecido à 180 graus, por 40min;



Algoritmos

Verifica se o número é par:

```
def check_if_number_is_odd (number: int):  
    if number % 2 == 0:  
        print("O número é par")  
    else:  
        print("O número é ímpar")
```



2026

Algoritmos



*Link para o livro no
Archive*



Variáveis

O que são?

Variáveis são uma forma literal de salvarmos memória. Podem ter diferentes ***escopos****

*: Lembrar do domínio de uma função na matemática.



Variáveis

Tipos de variáveis:

- n: **int**; #numero sem virgula
- raio: **double**; #numero com virgula
- nome: **str**; #sequencia de caracteres
-



Funções

Anatomia de uma função

def <nome_da_função> (*parâmetros*):

... lógica ...

obs.: o nome da função deve ser um frase que indica o que a função faz.

Exemplo: *calcular_raio_do_circulo*



Funções

Exemplos de funções

```
def calcula_area_triangulo(base: float, altura: float):  
    area = (base * altura) / 2  
    print(f"A área do triângulo é: {area}")  
  
def converte_temperatura_fahrenreit(graus_celsius: float):  
    temperatura = (graus_celsius * (9/5)) + 32  
    print(f"A temperatura em fahrenreit é: {temperatura}")  
  
def converte_em_dolares(reais: float):  
    dolares = reais / 5.04  
    print(f"O valor em dólares é: {dolares}")
```



Funções

Sobre funções

Funções são ao mesmo tempo a menor e mais básica parte da computação. Podem calcular o troco para uma compra, ou calcular a curvatura da luz em um buraco negro.

Mas para isso é importante vermos algumas estruturas de dados e um conjunto de operações.



Funções

Sobre funções

Funções são ao mesmo tempo a menor e mais básica parte da computação. Podem calcular o troco para uma compra, ou calcular a curvatura da luz em um buraco negro.

Mas para isso é importante vermos algumas estruturas de dados e um conjunto de operações.

Funções

retornos

Na maioria dos casos as funções podem ter seus resultados passados para variáveis, como o exemplo:

```
def say_hello():  
    return "Olá, mundo"
```

```
hello = say_hello()
```

Condicionais

João Augusto T. Laner

Unisenac
Centro Universitário RS



Fecomércio
IFEP
Sesc
Sindicatos Empresariais



Condicionais

O que são?

Condicionais são pequenos blocos que avaliam fluxo e controlam o prosseguimento do código. Vejamos um exemplo?

```
def verificar_se_pode_beber(idade: int):
```

```
    if idade > 18:
```

```
        return "pode beber"
```

```
    return "idade insuficiente"
```



Condicionais

Operadores comparativos

Para que possamos condicionar o fluxo de um algoritmo, alguns operadores se fazem necessários, são eles:

>, maior

>=, maior igual

<, menor

<=, menor igual

==, igual

!=, diferente



Condicionais

Operadores lógicos

Operadores lógicos atuam como complementos de operadores comparativos e servem para comparar de forma **literal** os objetos.

Condicionais

Operadores lógicos (literais)

and	se x e y, então...
or	se x ou y, então...
is	se x é y, então...
not	se x não é y, então...



Condicionais

Operadores lógicos

Operadores lógicos atuam como complementos de operadores comparativos e servem para comparar **literalmente** os objetos.

EXEMPLO

Exemplo 2

Funções

Você quer automatizar a entrada para um clube chamado **SEU NOME**. Para isso crie uma função que verifica o nome **e** a idade do membro, se o nome for igual e a idade for maior ou igual a idade de você, então ele é membro do clube.



Exemplo 2

Funções

```
def verifica_membro(nome: str, idade:int):
```



Exemplo 2

Funções

```
def verifica_membro(nome: str, idade:int):  
    if nome == "João" and idade >= 28:
```



Exemplo 2

Funções

```
def verifica_membro(nome: str, idade:int):  
    if nome == "João" and idade >= 28:  
        return True  
    return False
```



Condicionais

Exemplo

```
15
16 def verifica_membro_do_clube(nome:str, idade:int) -> str:
17     if nome != "João" and idade < 28:
18         return "Você não é membro do clube"
19
20     return "Bem vindo ao clube!"
21
```

Listas (Arrays)

João Augusto T. Laner

Unisenac
Centro Universitário RS



Fecomércio
IFEP
Sesc
Sindicatos Empresariais



Listas

Definição

Listas ou **arrays** são sequências de algum tipo de dado e são declarados usando os colchetes ou o objeto **list()** presente na biblioteca padrão. e inclusa na linguagem.

Listas

Definição

Na computação, a partir dos arrays, derivam-se várias outras estruturas, como filas e pilhas.



Listas

Declaração

```
numeros = [1,2,3,4,5,6] #lista contendo os números 1, 2, 3, 4, 5 e 6  
salada_de_fruta = ["abacaxi", "mamão", "pera", "maçã"]  
pessoas = [{'nome':'joão', 'idade':28}, {'nome':'pyetra', 'idade':28}]  
#lista contendo 2 estruturas dicionário
```

O que podemos fazer com uma lista?

João Augusto T. Laner

Unisenac
Centro Universitário RS



Fecomércio
IFEP
Sesc
Sindicatos Empresariais



O que podemos fazer?

Frutas

Se tivermos uma tigela de frutas, o que podemos fazer?

- ver todas as frutas da tigela;
- remover todas as frutas;
- remover somente um tipo de fruta;
- adicionar outra fruta;

O que podemos fazer?

Frutas

- dividir as frutas em duas tigelas menores;
- mover somente as frutas de um determinado tipo para outra tigela;
- remover a fruta do topo;
- remover a fruta mais do fundo;



O que podemos fazer?

Frutas

- adicionar uma fruta num local específico da tigela;
- ordenar as frutas por cor;
- ordenar as frutas por tamanho;
- ordenar as frutas por sabor;
- ...

Loops

João Augusto T. Laner

Unisenac
Centro Universitário RS



Fecomércio
IFEP
Sesc
Sindicatos Empresariais

Loops

Definição

Loops são estruturas de código que nos permitem percorrer listas e acessar seus valores. Existem **2** estruturas principais para isso: o **for** e o **while**



Loops

For - exemplo

```
salada_de_fruta = ["abacaxi", "mamão", "pera", "maçã"]
```

```
for fruta in salada_de_fruta:  
    print(fruta)
```



Loops

While - exemplo

```
salada_de_fruta = ["abacaxi", "mamão", "pera", "maçã"]
```

```
index = 0
```

```
while (index < len(salada_de_fruta)):
```

```
    print(salada_de_fruta[index])
```

```
    index+=1
```



2026

Criando nossa cesta de Frutas Em python...

Bora pro código!!!!

referências:

<https://docs.python.org/pt-br/3.14/tutorial/datastructures.html>

[#more-on-lists](#)

Classes

João Augusto T. Laner

Unisenac
Centro Universitário RS



Fecomércio
IFEP
Sesc
Sindicatos Empresariais



Classes

Definição

Classes, na computação, são uma forma de abstrairmos o mundo real para dentro do computador. Essa técnica é muito poderosa e nos permite criar, dentro dos limites, representações de **objetos** ou **eventos** (dos mais variados)



Classes

Definição

Usaremos classes ao longo do curso todo. Assim como neste módulo, usaremos para representar entidades do nosso sistema na unidade de back-end



Classes

Declaração

```
class Gato:
```

```
    pelagem:str = "preto"
```

```
    porte:str = "pequeno"
```

```
    nome:str = "Morcega"
```

```
    idade:int = 2
```



Classes

Declaração

```
class Gato:
```

```
    pelagem: str = "preto"
```

```
    porte: str = "pequeno"
```

```
    nome: str = "Morcega"
```

```
    idade: int = 2
```





Classes

Inicialização

As vezes, é necessário que criemos classes com valores que são definidos arbitrariamente (ex.: um cadastro de produto)

Para isso usaremos o método/função `__init__()`:



Classes

Inicialização

As vezes, é necessário que criemos classes com valores que são definidos arbitrariamente (ex.: um cadastro de produto)

Para isso usaremos o método/função `__init__()`:



Classes

`__init__`

```
2 class Gato:
3
4     pelagem:str = ""
5     porte:str = ""
6     nome:str = ""
7     idade:int = 0
8
9
10    def __init__(self, pelagem:str, porte:str, nome:str, idade:int):
11        self.pelagem: str = pelagem
12        self.porte: str = porte
13        self.nome: str = nome
14        self.idade: int = idade
15
```

Classes

Atributos

Classes também podem ser pensadas como **wrappers** de atributos. Por exemplo, se eu quiser acessar a cor de um gato, preciso acessar a classe antes.



Classes

Acessando atributos

```
gato = Gato("preto", "pequeno", "morcega", 2) #criando classe
```

```
# ou
```

```
gato = Gato(nome="morcega", idade=2, porte="pequeno",  
pelagem="preto")
```



Classes

Acessando atributos

```
gato = Gato("preto", "pequeno", "morcega", 2) #criando classe
```

```
def mostraNome(self):  
    print(f"O nome do gato é {gato.nome}")
```

Classes

Métodos

Classes também podem ter funções (**métodos**) próprios, como por exemplo:



Classes

Métodos

```
class Gato:
```

```
...
```

```
def miar(self): #self referencia a propria classe
```

```
    print("Miau!")
```



Classes

Modularidade

Como já vimos anteriormente, podemos definir classes como módulos (pedaços) de outros códigos, que podem ser reutilizados.



Classes

Modularidade

Se tivermos o arquivo ***gato.py*** com a classe, podemos importar o código dentro do outro arquivo usando ***import***

```
from gato import Gato
```

Herança e Polimorfismo

João Augusto T. Laner

Unisenac
Centro Universitário RS



Fecomércio
IFEP
Sesc
Sindicatos Empresariais



Classes

Herança e Polimorfismo

Objetos podem ter relações entre si, como por exemplo, ser uma espécie de subdivisão de outro. Vejamos a classe **Character**

A partir dessas subdivisões, as classes filhas podem ser **mais especializadas** que a classe mãe, exigindo variações dos métodos.



Character

Classe mãe

```
class Character:

    nome:str
    vida: float
    level:int = 1

    def __init__(self, nome:str, vida: float):
        self.nome: str = nome
        self.vida: float = vida
```



Classes

Warrior & Wizard

Se pensarmos em um jogo online, a classe **Character** é a base de qualquer outro personagem, então vejamos agora as classes **Guerreiro** e **Mago**



Guerreiro

Implementação

```
1 from character import Character
2
3 class Warrior(Character):
4
5     def __init__(self, nome:str):
6         super().__init__(nome, vida = 150)
7
```



Classe

Guerreiro (Warrior)

Implementação de Herança

```
1 from character import Character
2
3 class Warrior(Character):
4
5     def __init__(self, nome:str):
6         super().__init__(nome, vida = 150)
7
```





Classe

Mago (Mage)

```
1 from character import Character
2
3 class Mage(Character):
4
5     def __init__(self, nome:str):
6         super().__init__(nome, vida = 90)
7
```



Classes

Warrior & Wizard

Por serem personagens diferentes, mas com mesmas possibilidades (ambos podem atacar) é interessante usar a função com **lógicas diferentes**



Guerreiro (Warrior)

```
1 from character import Character
2
3 class Warrior(Character):
4
5     def __init__(self, nome:str):
6         super().__init__(nome, vida = 150)
7
8     def atacar(self, inimigo: Character):
9         inimigo.vida -= 20
```



Mago (Mage)

```
1 from character import Character
2
3 class Mage(Character):
4
5     def __init__(self, nome:str):
6         super().__init__(nome, vida = 90)
7
8     def atacar(self, inimigo: Character) -> None:
9         if super().vida <= 40:
10             inimigo.vida -= 40
11         else:
12             inimigo.vida -= 25
```

Erros e Try/Except

João Augusto T. Laner

Unisenac
Centro Universitário RS



Fecomércio
IFEP
Sesc
Sindicatos Empresariais



Erros

Funcionalidade

É comum usarmos erros quando precisamos verificar uma situação antes de prosseguir.

Ex. Abrir a porta > precisamos da chave



Erros

Como aparecem

Se tivermos uma classe que realiza uma divisão, podemos **levantar** um erro:

```
def dividir(self, divisor:int):  
    if divisor == 0: #se divisor for zero  
        raise ZeroDivisionError('Não pode dividir por zero')
```